

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет
по учебной практике (ознакомительной)
по теме
СПРАВКА АЭРОПОРТА

Студент:
гр. 558301 Минкевич А. С.

Руководитель:
старший преподаватель
Ковальчук А. М.

МИНСК 2026

СОДЕРЖАНИЕ

УСЛОВИЕ ЗАДАНИЯ	3
ВВЕДЕНИЕ	4
1 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ	5
1.1 Входные и выходные данные	5
2 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ	6
2.1 Разработка схем алгоритмов	6
2.2 Разработка алгоритмов	6
3 РЕЗУЛЬТАТ РАБОТЫ	8
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11
ПРИЛОЖЕНИЕ А	12
ПРИЛОЖЕНИЕ Б	14
ПРИЛОЖЕНИЕ В	16
ПРИЛОЖЕНИЕ Г	21
ПРИЛОЖЕНИЕ Д	49

УСЛОВИЕ ЗАДАНИЯ

Напісаць праграму для кіравання інфармацыяй аб авіярэйсах аэрапорта. Праграма павінна выдаваць даведку пра маршрут, гэта значыць час рэйса, статус, тэрмінал і мадэль самалёта. Сістэма павінна дазваляць дадаваць, рэдагаваць і выдаляць рэйсы (з магчымасцю адмены апошняга дзеяння).

ВВЕДЕНИЕ

Аўтаматызацыя працэсаў уліку і маніторынгу авіярэйсаў з'яўляецца важнай задачай для любога сучаснага аэрапорта. Для забеспячэння хуткай працы і аптымальнага выкарыстання сістэмных рэсурсаў такія праграмныя прадукты патрабуюць дбайнага праектавання структур дадзеных і алгарытмаў апрацоўкі інфармацыі.

Мэтай дадзенай працы з'яўляецца распрацоўка кансольнага прыкладання на мове праграмавання C, якое рэалізуе базавы функцыянал інфармацыйнага табло аэрапорта з гнуткай сістэмай фільтрацыі і пошуку.

Для дасягнення пастаўленай мэты неабходна вырашыць наступныя задачы: спраектаваць структуры дадзеных для захоўвання інфармацыі пра рэйсы; рэалізаваць дынамічныя структуры дадзеных (стэк, чаргу і бінарнае дрэва пошуку); распрацаваць сістэму захавання і загрузкі базы дадзеных з выкарыстаннем тэкставага і бінарнага файлавага ўводу-вываду; рэалізаваць кансольны карыстальніцкі інтэрфейс з размеркаваннем правоў доступу.

У першым раздзеле тлумачальнай запіскі праводзіцца праектаванне структур дадзеных. У другім раздзеле апісваецца працэс распрацоўкі алгарытмаў і праграмных модуляў. Трэці раздзел прысвечаны дэманстрацыі вынікаў работы праграмы.

1 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

1.1 Входные и выходные данные

Для захоўвання інфармацыі ў дадатку распрацавана іерархічная мадэль дадзеных. У табліцы 1.1 прадстаўлены склад і ўкладзенасць структур, якія выкарыстоўваюцца для апісання рэйсаў і ўсёй сістэмы аэрапорта.

Таблица 1.1 — Іерархія і склад структур дадзеных дадатку

Структуры і тыпы дадзеных		
enum FlightType	ARRIVAL (0)	
	DEPARTURE (1)	
enum FlightStatus	ON_TIME, DELAYED	
	BOARDING, DEPARTED	
	LANDED, CANCELED	
	(Цэлалікавыя значэнні ад 0 да 5)	
struct DateTime	int day, month, year	
	int hour, minute	
struct Flight	char flightNumber[10]	
	char airline[50], city[50]	
	FlightType type	
	DateTime scheduleTime, actualTime	
	FlightStatus status	
	char terminal, int gate	
struct StackNode	Flight data	
	struct StackNode* next	
struct QueueNode	Flight data	
	struct QueueNode* next	
struct TreeNode	Flight data	
	struct TreeNode* left	
	struct TreeNode* right	
struct AirportSystem	Flight flights[MAX_FLIGHTS]	
	int count	
	Stack history	StackNode* top
		int size
	Queue pendingQueue	QueueNode* head, tail
		int size
TreeNode* searchTree		

2 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

2.1 Разработка схем алгоритмов

Схема алгоритму асноўнай праграмы прадстаўлена ў дадатку А. Яна ўключае цыкл аўтэнтыфікацыі і пераход да галоўнага меню.

Функцыя `sys_delete_flight` дазваляе бяспечна выдаляць рэйсы з захаваннем стану ў стэку для наступнай адмены (схема прадстаўлена ў дадатку Б).

Функцыя `sys_sort_by_schedule` дазваляе сартаваць масіў рэйсаў па часе метадам устаўкі.

2.2 Разработка алгоритмов

Функцыя `sys_delete_flight` выконвае выдаленне элемента з масіва сістэмы з папярэднім захаваннем стану ў стэк гісторыі і выдаленнем адпаведнага вузла з бінарнага дрэва пошуку.

Перадаваемыя параметры:

`sys` — указальнік на структуру сістэмы (`AirportSystem`).

`idx` — цэлалікавае значэнне індэкса рэйса для выдалення.

Крок 1. Пачатак.

Крок 2. Аб'явіць лакальную зменную `i` (цэлага тыпу) для параметру цыклу.

Крок 3. Калі індэкс `idx` меншы за 0 або большы/роўны значэнню `sys->count`, перарваць выкананне функцыі.

Крок 4. Выклікаць функцыю `stack_push(&sys->history, sys->flights[idx])` для захавання бягучага стану рэйса ў гісторыю.

Крок 5. Прысвойце зменнай `sys->searchTree` вынік выканання функцыі `bst_delete`.

Крок 6. Цыкл з параметрам `i` ад значэння `idx` да `sys->count - 1`.

Крок 7. Прысвойце элементу масіва `sys->flights[i]` значэнне наступнага элемента масіва `sys->flights[i + 1]`.

Крок 8. Канец цыклу па `i`.

Крок 9. Паменшыце значэнне зменнай `sys->count` на 1.

Крок 10. Канец.

Функцыя `sys_sort_by_schedule` выконвае сартаванне масіва рэйсаў па часе вылету/прылёту (`scheduleTime`) з выкарыстаннем алгарытму сартавання ўстаўкай.

Крок 1. Пачатак.

Крок 2. Аб'явіць лакальныя зменныя `i` і `j` (цэлага тыпу) для цыклаў, а таксама `key` (структура тыпу `Flight`) для часовага захавання элемента.

Крок 3. Цыкл з параметрам `i` ад 1 да `sys->count`.

Крок 4. Прысвойце зменнай `key` значэнне элемента масіва `sys->flights[i]`.

Крок 5. Прысвойце зменнай `j` значэнне выразу `i - 1`.

Крок 6. Цыкл пакуль `j >= 0` і функцыя `datetime_compare(...)` вяртае лік, большы за 0.

Крок 7. Прысвойце элементу `sys->flights[j + 1]` значэнне `sys->flights[j]`.

Крок 8. Паменшыце значэнне зменнай `j` на 1.

Крок 9. Канец цыклу пакуль.

Крок 10. Прысвойце элементу масіва `sys->flights[j + 1]` значэнне зменнай `key`.

Крок 11. Канец цыклу па `i`.

Крок 12. Канец.

3 РЕЗУЛЬТАТ РАБОТЫ

Ніжэй прыведзены скрыншоты, якія дэманструюць асноўны функцыянал распрацаванага дадатку.

```
=====
      AIRPORT INFORMATION SYSTEM
=====
User: admin [ADMIN]
Flights on board: 8 | Pending: 0 | History stack: 0

1. View all flights
2. Search / filter flights
3. Sort by schedule
4. BST index (search by number)
== Admin ==
5. Add flight
6. Edit flight
7. Delete flight
8. Undo last change
9. Pending queue management
10. Load more data (input)
== File ==
11. Save / export
0. Exit
```

Рисунок 3.1 — Галоўнае меню праграмы

```
=== Add New Flight ===
Flight number (e.g. B2-737): B2-737
Airline: Belavia
City (destination/origin): Minsk
Aircraft model: Flew-3
Type (0=Arrival, 1=Departure): 1
Scheduled time:
  Day   (1-31): 2
  Month (1-12): 3
  Year:           2026
  Hour   (0-23): 6
  Minute(0-59): 23
Actual time (same if on schedule):
  Day   (1-31): 2
  Month (1-12): 3
  Year:           2026
  Hour   (0-23): 6
  Minute(0-59): 23
Status:
  0=On time  1=Delayed  2=Boarding  3=Departed  4=Landed  5=Canceled
```

Рисунок 3.2 — Увод інфармацыі аб новым рэйсе

No.	Flight	T	Airline	City	Aircraft	Scheduled	Actual	Status	T/G
1	B2-737	DEP	Belavia	Minsk	B737-800	25.05 14:00	25.05 14:00	On time	A5
2	LH-1234	ARR	Lufthansa	Frankfurt	A320	25.05 09:30	25.05 09:50	Delayed	B12
3	FR-9901	DEP	Ryanair	Dublin	B737-MAX	25.05 11:00	25.05 11:00	Boarding	C3
4	SU-1502	ARR	Aeroflot	Moscow	SU-100	25.05 16:45	25.05 16:45	On time	A7
5	QR-421	DEP	Qatar Airways	Doha	A380	25.05 22:10	25.05 23:30	Delayed	B1
6	TK-009	ARR	Turkish Airlines	Istanbul	B777	25.05 07:15	25.05 07:15	Landed	C9
7	W6-3302	DEP	Wizz Air	Budapest	A321neo	25.05 13:20	25.05 13:20	On time	A4
8	EK-502	ARR	Emirates	Dubai	A380	25.05 19:05	25.05 19:05	On time	B2
9	B2-737	DEP	Belavia	Minsk	Flew-3	02.03 06:23	02.03 06:23	On time	A2
Total: 9 flight(s).									

Рисунок 3.3 — Вывад поўнага спіса рэйсаў у выглядзе табліцы

```

=== Search Flights ===
Filter by type?      (1=yes, 0=no): 1
      0=Arrival, 1=Departure: 1
Filter by city?      (1=yes, 0=no): 0
Filter by status?    (1=yes, 0=no):
0
Filter by date/time (1=yes, 0=no): 0

--- Search Results ---

```

Рисунок 3.4 — Пошук па фільтрах у сістэме

ЗАКЛЮЧЕНИЕ

Падчас выканання практыкі была паспяхова распрацавана кансольная інфармацыйная сістэма аэрапорта. Дадатак дае карыстальнікам дакладную інфармацыю пра час рэйсаў, статусы вылетаў і прылётаў, нумары тэрміналаў і мадэлі самалётаў.

Праграма мае зручны і зразумелы інтэрфейс, які падзяляе правы доступу паміж пасажырамі і адміністратарамі. Выкарыстанне дынамічных структур дадзеных (бінарнае дрэва пошуку, стэк для гісторыі адмен і чарга для чакаючых рэйсаў) значна павысіла эфектыўнасць апрацоўкі інфармацыі.

Асаблівая ўвага пры распрацоўцы была нададзена надзейнасці ўводу дадзеных і бяспечнай працы з памяццю. Файлавы ўвод-вывад дазваляе бяспечна захоўваць базу дадзеных паміж сесіямі ў розных фарматах.

У выніку выканання праекта былі не толькі замацаваны навыкі праграмавання на мове C, але і атрыманы каштоўны вопыт стварэння рэальнага прыкладання для працы з транспартнымі маршрутамі і складанымі структурамі дадзеных.

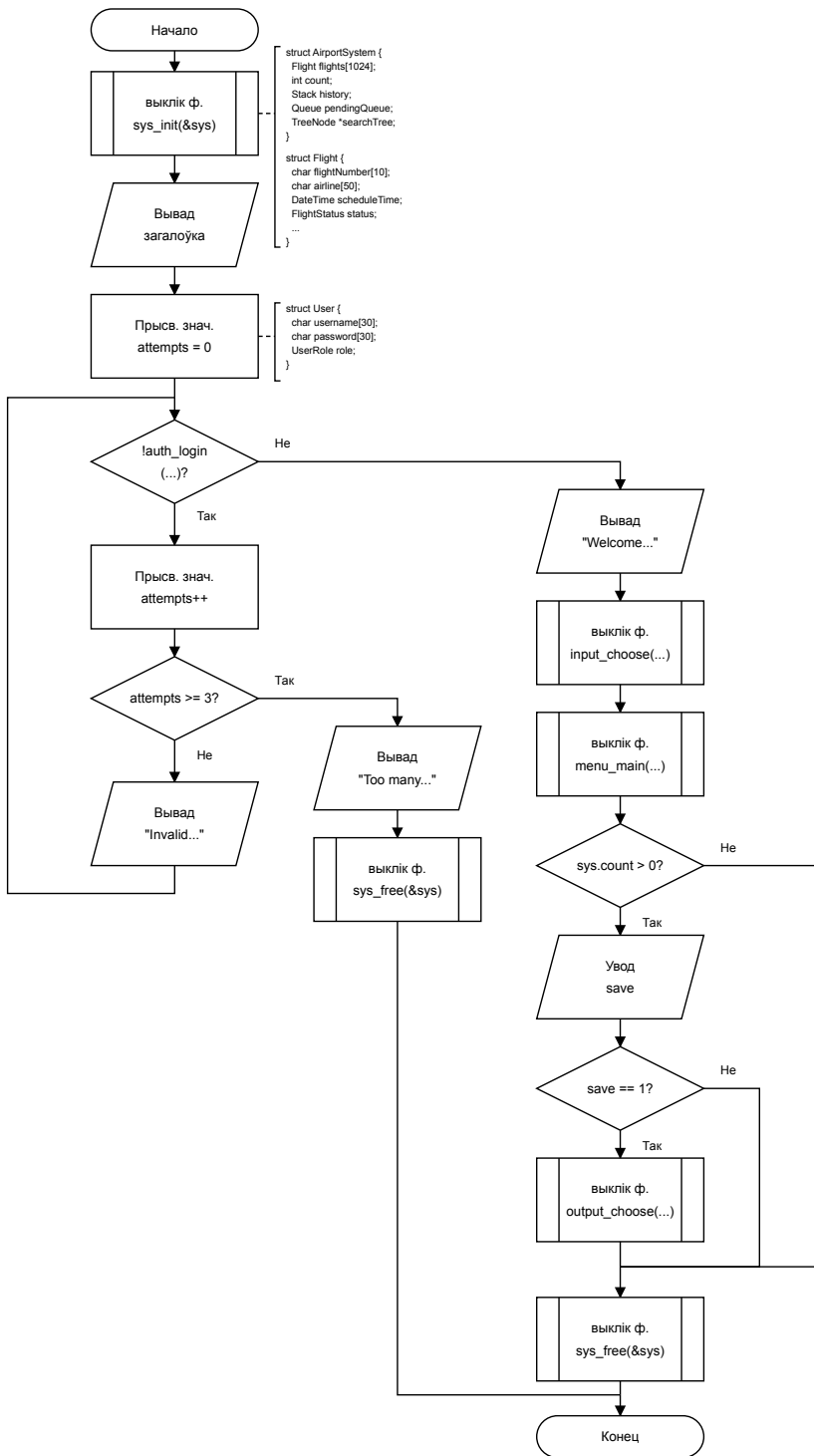
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Основы алгоритмизации и программирования : лаборатор. практикум для студентов специальности 1-40 02 01 «Вычисл. машины, системы и сети» всех форм обучения. В 2 ч. Ч. 2 / сост. Ю. А. Луцик [и др.]. – Минск : БГУИР, 2010. – 36 с. : ил.

[2] Шилдт, Г. С: полное руководство, 4-е издание / Г. Шилдт. – М. : Вильямс, 2011. – 704 с.

[3] StackOverflow [Электронный ресурс] – Вопросы – режим доступа: <https://ru.stackoverflow.com>.

ПРИЛОЖЕНИЕ А
(обязательное)
Схема алгоритма программы main

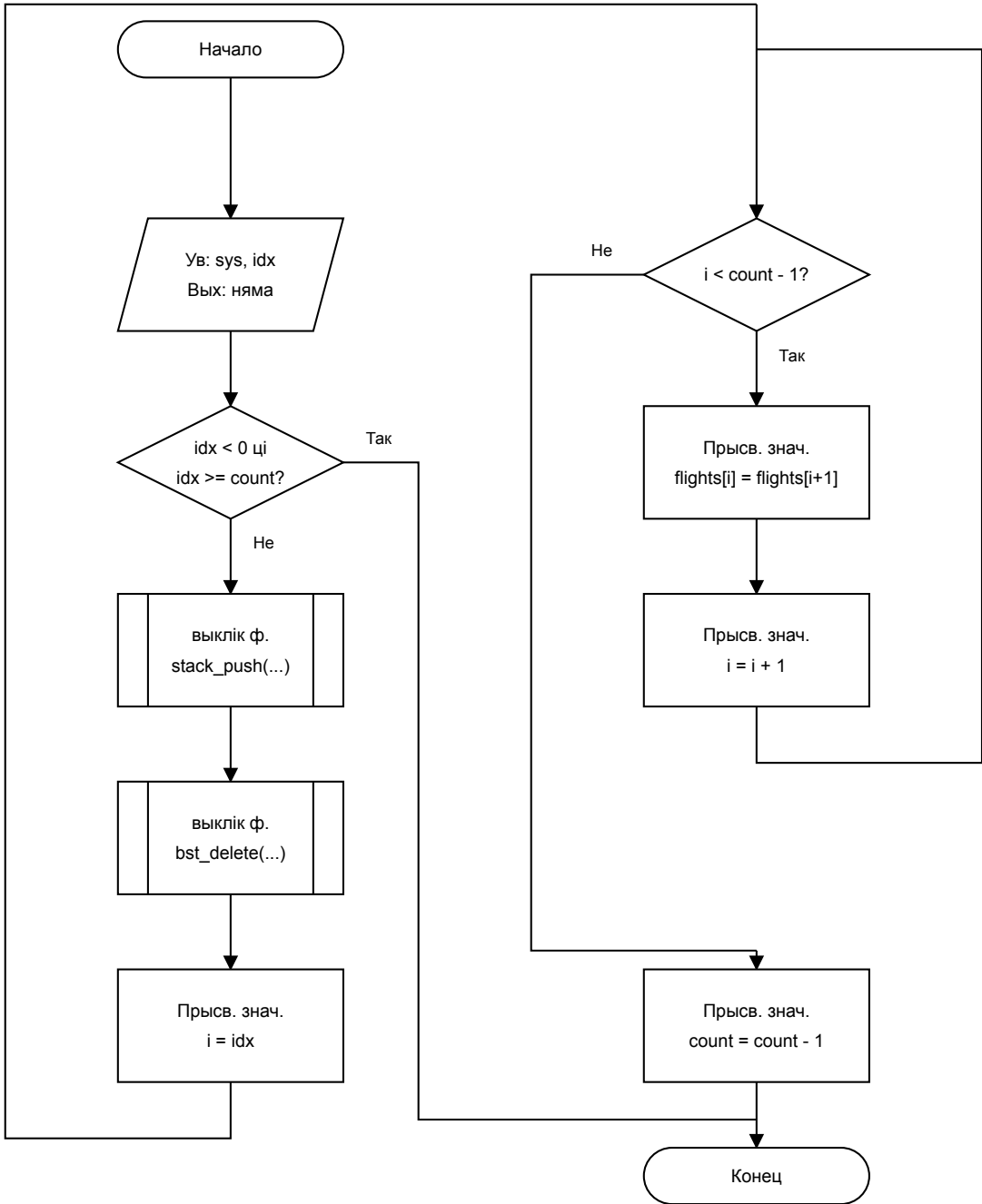


					ГЧИР.6-05-0611-05.121				
					Блок-схема функции main	Лист.		Масса	Масштаб
Изм.	Лист	№ докум.	Подпись	Дата			Т		
Разраб.		Минкевич							
Проб.		Ковальчук							
						Лист		Листов 1	
						ЭВМ, зр. 558301			

ПРИЛОЖЕНИЕ Б

(обязательное)

Схема алгоритма функции sys_delete_flight



					ГУИР.6-05-0611-05.121				
					Блок-схема функции sys_delete_flight	Лист.		Масса	Масштаб
Изм.	Лист	№ докум.	Подпись	Дата		Т			
Разраб.	Минкевич								
Проб.	Ковальчук								
						Лист		Листов 1	
					ЭВМ, зр. 558301				

ПРИЛОЖЕНИЕ В

(обязательное)

Код файла header.h

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#ifndef HEADER_H
#define HEADER_H

// ===== КАНСТАНТЫ =====
#define MAX_FILENAME 100
#define MAX_FLIGHTS 1024 //
Максимальная колькасць рэйсаў у сістэме.

// ===== ПЕРАЛІЧЭННІ =====

typedef enum { FALSE = 0, TRUE = 1 } Boolean; // Уласны
булеан, бо ў C89/C90 яго няма.

typedef enum {
    ARRIVAL = 0, // Прылёт.
    DEPARTURE = 1, // Вылет.
} FlightType;

typedef enum {
    ON_TIME = 0, // Па
    раскладзе.
    DELAYED = 1, //
    Спазняяцца.
    BOARDING = 2, // Пасадка
    (вылет) .
    DEPARTED = 3, // Ужо
    вылецеў.
    LANDED = 4, // Ужо
    прызямліўся.
    CANCELED = 5 //
    Адменены.
} FlightStatus;

typedef enum {
    ROLE_PASSENGER = 0, // Толькі
    прагляд.
    ROLE_ADMIN = 1 // Поўны
    доступ.
```



```

} UserRole;

// ===== СТРУКТУРЫ =====

typedef struct {
    char    username[30];           // Логін.
    char    password[30];          // Пароль.
    UserRole role;                  //
    // Урoвень доступу.
} User;

typedef struct {                   // Час
    // захоуваєцца асобнымі палямі — зручна для сартавання.
    int day;
    int month;
    int year;
    int hour;
    int minute;
} DateTime;

typedef struct {
    char    flightNumber[10];       // Нумар
    // рэйса, напрыклад "B2-737".
    char    airline[50];            //
    // Авіякампанія.
    char    city[50];               // Горад
    // прылёту або вылету.
    char    airplaneModel[30];      // Мадэль
    // самалёта.

    FlightType type;                // Прылёт
    // або вылет.
    DateTime scheduleTime;          // Час па
    // плане.
    DateTime actualTime;            // Рэальны
    // час з улікам затрымкі.
    FlightStatus status;            // Бягучы
    // стан.

    char    terminal;               // Літара
    // тэрмінала (A, B, C...).
    int     gate;                   // Нумар
    // гейта.
} Flight;

typedef struct {                   // Фільтр
    // для разумнага пошуку.
    Boolean useTypeFilter;

```

```

    FlightType targetType;

    Boolean    useCityFilter;
    char       targetCity[50];

    Boolean    useStatusFilter;
    FlightStatus targetStatus;

    Boolean    useTimeFilter;
    DateTime   startTime;
} SearchFilter;

// ===== СТЭК =====

typedef struct StackNode {
    Flight      data;
    struct StackNode *next;
} StackNode;

typedef struct {
    StackNode *top;
    int       size;
} Stack;

// ===== ЧАРГА =====

typedef struct QueueNode {
    Flight      data;
    struct QueueNode *next;
} QueueNode;

typedef struct {
    QueueNode *head;
    QueueNode *tail;
    int       size;
} Queue;

// ===== БИНАРНАЕ ДРЭВА =====

typedef struct TreeNode {
    Flight      data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

// ===== АСНОЎНАЯ СИСТЭМА =====

typedef struct {

```

```

    Flight flights[MAX_FLIGHTS]; // Асноўны
масіў усіх рэйсаў.
    int count; //
Колькасць рэйсаў зараз.
    Stack history; // Стэк
для адмены дзеянняў.
    Queue pendingQueue; // Чарга
рэйсаў на пацвярджэнне.
    TreeNode *searchTree; // BST для
хуткага пошуку.
} AirportSystem;

// ===== ПРАТАТЫПЫ ФУНКЦЫЙ =====

void read_line(char *buf, int size, FILE *f);
void rewind_stdin(void);
void GetInt(int *value);
void GetChar(char *value);
void clear_screen(void);
void press_enter(void);

void datetime_print(DateTime dt);
int datetime_compare(DateTime a, DateTime b);
void datetime_input(DateTime *dt);

const char *flight_type_str(FlightType t);
const char *flight_status_str(FlightStatus s);
FlightType parse_flight_type(const char *s);
FlightStatus parse_flight_status(const char *s);

void flight_print_header(void);
void flight_print_row(int idx, const Flight *f);
void flight_input_keyboard(Flight *f);
int flight_parse_line(const char *line, Flight *f);
Boolean flight_matches_filter(const Flight *f, const SearchFilter
*filter);

void stack_init(Stack *s);
void stack_push(Stack *s, Flight f);
Boolean stack_pop(Stack *s, Flight *out);
Boolean stack_peek(const Stack *s, Flight *out);
void stack_free(Stack *s);

void queue_init(Queue *q);
void queue_enqueue(Queue *q, Flight f);
Boolean queue_dequeue(Queue *q, Flight *out);
void queue_free(Queue *q);
void queue_print(const Queue *q);

```

```

TreeNode    *bst_insert(TreeNode *root, Flight f);
TreeNode    *bst_search(TreeNode *root, const char *flightNumber);
TreeNode    *bst_delete(TreeNode *root, const char *flightNumber);
void        bst_inorder_print(TreeNode *root);
void        bst_free(TreeNode *root);

void        sys_init(AirportSystem *sys);
void        sys_free(AirportSystem *sys);
void        sys_add_flight(AirportSystem *sys, Flight f);
Boolean     sys_find_by_number(AirportSystem *sys, const char *num,
int *outIdx);
void        sys_delete_flight(AirportSystem *sys, int idx);
void        sys_update_status(AirportSystem *sys, int idx,
FlightStatus newStatus);
void        sys_sort_by_schedule(AirportSystem *sys);
void        sys_print_all(AirportSystem *sys);
void        sys_search(AirportSystem *sys, const SearchFilter
*filter);
void        sys_undo(AirportSystem *sys);

void        input_choose(AirportSystem *sys);
void        input_keyboard(AirportSystem *sys);
void        input_txt_file(AirportSystem *sys);
void        input_binary_file(AirportSystem *sys);

void        output_choose(AirportSystem *sys);
void        output_txt_file(AirportSystem *sys, const char
*filename);
void        output_binary_file(AirportSystem *sys, const char
*filename);

void        menu_main(AirportSystem *sys, User *currentUser);
void        menu_search(AirportSystem *sys);
void        menu_add(AirportSystem *sys);
void        menu_edit(AirportSystem *sys);
void        menu_delete(AirportSystem *sys);
void        menu_pending_queue(AirportSystem *sys);
void        menu_bst(AirportSystem *sys);

Boolean     auth_login(User *out);

#endif

```

Листинг 1 — Исходный код заголовочного файла header.h

ПРИЛОЖЕНИЕ Г
(обязательное)
Код файла func.c

```
#include "header.h"

//
=====

//  УТИЛИТЫ
//
=====

void read_line(char *buf, int size, FILE
*f)                                // Бяспечнае чытанне радка з файла.
{
    if (fgets(buf, size, f) != NULL)
        buf[strcspn(buf, "\n")] =
'\0';                                // Выдаляем \n калі ёсць.
}

void
rewind_stdin(void)                //
Ачыстка буфера стандартнага ўводу.
{
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void GetInt(int
*value)                            // Бяспечны
ўвод цэлага ліку.
{
    while (1)
    {
        if (scanf("%d", value) == 1) { rewind_stdin(); return; }
        printf(" Invalid input. Please enter an integer: ");
        rewind_stdin();
    }
}

void GetChar(char
*value)                            // Бяспечны
ўвод аднаго сімвала.
{
    while (1)
    {
        if (scanf(" %c", value) == 1) { rewind_stdin(); return; }
```

```

        printf("  Invalid input. Please enter a character: ");
        rewind_stdin();
    }
}

void
clear_screen(void)                                     //
Ачыстка тэрміналу.
{
    printf("\033[2J\033[H");
}

void
press_enter(void)                                     //
Паўза — чакаем Enter ад карыстальніка.
{
    printf("\nPress Enter to continue...");
    rewind_stdin();
}

//
=====
//  DATETIME
//
=====

void datetime_print(DateTime
dt)                                                    // Вывад у фармаце
DD.MM.YYYY HH:MM.
{
    printf("%02d.%02d.%04d %02d:%02d",
            dt.day, dt.month, dt.year, dt.hour, dt.minute);
}

int datetime_compare(DateTime a, DateTime
b)                                                    // Параўнанне дат: вяртае -1, 0 або 1.
{
    if (a.year    != b.year)    return a.year    - b.year;
    if (a.month   != b.month)   return a.month   - b.month;
    if (a.day     != b.day)     return a.day     - b.day;
    if (a.hour    != b.hour)    return a.hour    - b.hour;
    return a.minute - b.minute;
}

void datetime_input(DateTime
*dt)                                                  // Інтэрактыўны ўвод даты
і часу.
{

```

```

    printf("    Day    (1-31): "); GetInt(&dt->day);
    printf("    Month  (1-12): "); GetInt(&dt->month);
    printf("    Year:      "); GetInt(&dt->year);
    printf("    Hour   (0-23): "); GetInt(&dt->hour);
    printf("    Minute (0-59): "); GetInt(&dt->minute);
}

//
=====

//  КАНБЕРТАРЫ
//
=====

const char *flight_type_str(FlightType
t)                                // Пераўтварэнне тыпу рэйса ў
радок.
{
    return (t == ARRIVAL) ? "ARR" : "DEP";
}

const char *flight_status_str(FlightStatus
s)                                // Пераўтварэнне статусу ў радок.
{
    switch (s)
    {
        case ON_TIME:    return "On time";
        case DELAYED:    return "Delayed";
        case BOARDING:   return "Boarding";
        case DEPARTED:   return "Departed";
        case LANDED:     return "Landed";
        case CANCELED:   return "Canceled";
        default:         return "Unknown";
    }
}

FlightType parse_flight_type(const char
*s)                                // Парсінг тыпу рэйса з радка.
{
    if (strcmp(s, "ARR") == 0 || strcmp(s, "ARRIVAL") == 0)    return
ARRIVAL;
    return DEPARTURE;
}

FlightStatus parse_flight_status(const char
*s)                                // Парсінг статусу рэйса з радка.
{
    if (strcmp(s, "On time") == 0) return ON_TIME;
    if (strcmp(s, "Delayed") == 0) return DELAYED;

```

```

        if (strcmp(s, "Boarding") == 0) return BOARDING;
        if (strcmp(s, "Departed") == 0) return DEPARTED;
        if (strcmp(s, "Landed") == 0) return LANDED;
        if (strcmp(s, "Canceled") == 0) return CANCELED;
        return ON_TIME;
    }

    //
    =====
    //  FLIGHT — ВЫБАД
    //
    =====

void
flight_print_header(void) //
Заголовак табліцы рэйсаў.
{
    printf("\n%-4s %-9s %-3s %-20s %-22s %-20s %-10s %-10s %-5s
%-4s\n",
        "No.", "Flight", "T", "Airline",
        "City", "Aircraft",
        "Scheduled", "Actual",
        "Status", "T/G");
    printf("%-4s %-9s %-3s %-20s %-22s %-20s %-10s %-10s %-10s
%-4s\n",
        "-----", "-----", "----", "-----",
        "-----", "-----",
        "-----", "-----",
        "-----", "----");
}

static void sprint_dt(char *buf, DateTime
dt) // Пераўтварэнне DateTime у радок для
табліцы.
{
    sprintf(buf, "%02d.%02d %02d:%02d",
        dt.day, dt.month, dt.hour, dt.minute);
}

void flight_print_row(int idx, const Flight
*f) // Вывад аднаго радка табліцы рэйсаў.
{
    char sched[20], actual[20];
    sprint_dt(sched, f->scheduleTime);
    sprint_dt(actual, f->actualTime);

    printf("%-4d %-9s %-3s %-20s %-22s %-20s %-16s %-16s %-10s
%c%-3d\n",

```



```

        idx + 1,
        f->flightNumber,
        flight_type_str(f->type),
        f->airline,
        f->city,
        f->airplaneModel,
        sched,
        actual,
        flight_status_str(f->status),
        f->terminal,
        f->gate);
    }

//
=====
//  FLIGHT - УВОД З КЛАВІЯТУРЫ
//
=====

void flight_input_keyboard(Flight
*f)                                     // Увод усіх полётів рейса з
клавіатури.
{
    printf("  Flight number (e.g. B2-737): ");
    read_line(f->flightNumber, sizeof(f->flightNumber), stdin);

    printf("  Airline: ");
    read_line(f->airline, sizeof(f->airline), stdin);

    printf("  City (destination/origin): ");
    read_line(f->city, sizeof(f->city), stdin);

    printf("  Aircraft model: ");
    read_line(f->airplaneModel, sizeof(f->airplaneModel), stdin);

    printf("  Type (0=Arrival, 1=Departure): ");
    int t; GetInt(&t);
    f->type = (t == 1) ? DEPARTURE : ARRIVAL;

    printf("  Scheduled time:\n");
    datetime_input(&f->scheduleTime);

    printf("  Actual time (same if on schedule):\n");
    datetime_input(&f->actualTime);

    printf("  Status:\n");
    printf("      0=On time  1=Delayed  2=Boarding  3=Departed  4=Landed
5=Canceled\n");
}

```

```

printf("  Choice: ");
int st; GetInt(&st);
if (st < 0 || st > 5) st = 0;
f->status = (FlightStatus)st;

printf("  Terminal (letter, e.g. A): ");
GetChar(&f->terminal);

printf("  Gate number: ");
GetInt(&f->gate);
}

//
=====
//  FLIGHT — ПАРСІНГ РАДКА З TXT ФАЙЛА
//
=====
//  Формат: FlightNum Type Airline City Aircraft Sched(DD.MM.YYYY/
HH:MM)
//          Actual(DD.MM.YYYY/HH:MM) Status Terminal Gate

int flight_parse_line(const char *line, Flight
*f) // Вяртае 1 пры поспеху, 0 пры памылцы.
{
    char typeStr[10], statusStr[15], schedStr[20], actualStr[20];

    // Формат: "B2-737 DEP Belavia Minsk B737-800 25.05.2026/14:00
25.05.2026/14:00 On_time A 5".
    int n = sscanf(line, "%9s %9s %49s %49s %29s %19s %19s %14s %c
%d",

                    f->flightNumber,
                    typeStr,
                    f->airline,
                    f->city,
                    f->airplaneModel,
                    schedStr,
                    actualStr,
                    statusStr,
                    &f->terminal,
                    &f->gate);

    if (n != 10) return 0;

    f->type = parse_flight_type(typeStr);

    sscanf(schedStr, "%d.%d.%d/%d:
%d", // Парсінг фармату "DD.MM.YYYY/
HH:MM".

```

```

        &f->scheduleTime.day, &f->scheduleTime.month, &f-
>scheduleTime.year,
        &f->scheduleTime.hour, &f->scheduleTime.minute);
    sscanf(actualStr, "%d.%d.%d/%d:%d",
        &f->actualTime.day, &f->actualTime.month, &f-
>actualTime.year,
        &f->actualTime.hour, &f->actualTime.minute);

    for (int i = 0; statusStr[i]; i+
+) // Замяняем падкрэсліванні
прабеламі ў statusStr.
        if (statusStr[i] == '_') statusStr[i] = ' ';

    f->status = parse_flight_status(statusStr);
    return 1;
}

//
=====
// FLIGHT - ФІЛЬТРАЦЫЯ
//
=====

Boolean flight_matches_filter(const Flight *f, const SearchFilter
*filter) // Праверка адпаведнасці рэйса фільтру.
{
    if (filter->useTypeFilter && f->type != filter->targetType)
        return FALSE;

    if (filter->useCityFilter &&
        strcmp(f->city, filter->targetCity) != 0)
        return FALSE;

    if (filter->useStatusFilter && f->status != filter->targetStatus)
        return FALSE;

    if (filter->useTimeFilter &&
        datetime_compare(f->scheduleTime, filter->startTime) < 0)
        return FALSE;

    return TRUE;
}

//
=====
// СТЭК
//
=====

```

```

void stack_init(Stack
*s)                                     // Ініціалізація
пустого стэка.
{
    s->top  = NULL;
    s->size = 0;
}

void stack_push(Stack *s, Flight
f)                                     // Даданне рэйса на вяршыню
стэка.
{
    StackNode *node = (StackNode *)malloc(sizeof(StackNode));
    if (node == NULL) { fprintf(stderr, "Fatal: stack malloc
failed\n"); exit(1); }
    node->data = f;
    node->next = s->top;
    s->top     = node;
    s->size++;
}

Boolean stack_pop(Stack *s, Flight
*out)                                 // Зняцце рэйса з вяршыні
стэка.
{
    if (s->top == NULL) return FALSE;
    StackNode *tmp = s->top;
    if (out) *out = tmp->data;
    s->top     = tmp->next;
    free(tmp);
    s->size--;
    return TRUE;
}

Boolean stack_peek(const Stack *s, Flight
*out)                                 // Прагляд вяршыні стэка без зняцця.
{
    if (s->top == NULL) return FALSE;
    if (out) *out = s->top->data;
    return TRUE;
}

void stack_free(Stack
*s)                                   // Ачыстка ўсяго
стэка.
{
    while (stack_pop(s, NULL));
}

```

```

}

//
=====
//  ЧАРГІ
//
=====

void queue_init(Queue
*q)                                     // Ініціялізація
пустой чаргі.
{
    q->head = q->tail = NULL;
    q->size = 0;
}

void queue_enqueue(Queue *q, Flight
f)                                     // Даданне рейса ў канец чаргі.
{
    QueueNode *node = (QueueNode *)malloc(sizeof(QueueNode));
    if (node == NULL) { fprintf(stderr, "Fatal: queue malloc
failed\n"); exit(1); }
    node->data = f;
    node->next = NULL;

    if (q->tail == NULL) { q->head = q->tail = node; }
    else                    { q->tail->next = node; q->tail = node; }
    q->size++;
}

Boolean queue_dequeue(Queue *q, Flight
*out)                                 // Выдаленне рейса з пачатку чаргі.
{
    if (q->head == NULL) return FALSE;
    QueueNode *tmp = q->head;
    if (out) *out = tmp->data;
    q->head      = tmp->next;
    if (q->head == NULL) q->tail = NULL;
    free(tmp);
    q->size--;
    return TRUE;
}

void queue_free(Queue
*q)                                   // Вызваленне ўсёй
памяці чаргі.
{
    while (queue_dequeue(q, NULL));
}

```

```

}

void queue_print(const Queue
*q) // Вывод чаргі рэйсаў на
экран.
{
    if (q->size == 0) { printf(" (queue is empty)\n"); return; }
    printf(" Pending queue (%d flights):\n", q->size);
    flight_print_header();
    int i = 0;
    QueueNode *cur = q->head;
    while (cur != NULL) { flight_print_row(i++, &cur->data); cur =
cur->next; }
}

//
=====
// BST (Binary Search Tree) – сартаванне па нумары рэйса
//
=====

TreeNode *bst_insert(TreeNode *root, Flight
f) // Устаўка рэйса ў дрэва.
{
    if (root == NULL)
    {
        TreeNode *node = (TreeNode *)malloc(sizeof(TreeNode));
        if (!node) { fprintf(stderr, "Fatal: bst malloc failed\n");
exit(1); }
        node->data = f;
        node->left = node->right = NULL;
        return node;
    }
    int cmp = strcmp(f.flightNumber, root->data.flightNumber);
    if (cmp < 0) root->left = bst_insert(root->left, f);
    else if (cmp > 0) root->right = bst_insert(root->right, f);
    else
        root->data =
f; // Абнаўляем існуючы вузел.
    return root;
}

TreeNode *bst_search(TreeNode *root, const char
*flightNumber) // Пошук вузла па нумары рэйса.
{
    if (root == NULL) return NULL;
    int cmp = strcmp(flightNumber, root->data.flightNumber);
    if (cmp < 0) return bst_search(root->left, flightNumber);
    else if (cmp > 0) return bst_search(root->right, flightNumber);
}

```

```

        return root;
    }

static TreeNode *bst_min_node(TreeNode
*root)                                // Знайсі вузел з мінімальным
ключом.
{
    while (root->left) root = root->left;
    return root;
}

TreeNode *bst_delete(TreeNode *root, const char
*flightNumber)                        // Видалення вузла з дрэва.
{
    if (root == NULL) return NULL;
    int cmp = strcmp(flightNumber, root->data.flightNumber);
    if (cmp < 0) root->left = bst_delete(root->left,
flightNumber);
    else if (cmp > 0) root->right = bst_delete(root->right,
flightNumber);
    else
    {
        if (root->left ==
NULL)                                // Вузел з адным або
нулявым нашчадкамі.
        {
            TreeNode *tmp = root->right;
            free(root);
            return tmp;
        }
        if (root->right == NULL)
        {
            TreeNode *tmp = root->left;
            free(root);
            return tmp;
        }
        TreeNode *succ = bst_min_node(root-
>right);                             // Два нашчадкі: замяняем значэннем
наступніка.
        root->data = succ->data;
        root->right = bst_delete(root->right, succ-
>data.flightNumber);
    }
    return root;
}

void bst_inorder_print(TreeNode
*root)                                // Вывад дрэва ў парадку

```

Ўзрастання нумара.

```
{
    if (root == NULL) return;
    bst_inorder_print(root->left);
    flight_print_row(0, &root->data);
    bst_inorder_print(root->right);
}
```

void bst_free(TreeNode

*root)

// Вызваленне ўсёй

памяці дрэва.

```
{
    if (root == NULL) return;
    bst_free(root->left);
    bst_free(root->right);
    free(root);
}
```

//

=====

// АСНОЎНАЯ СІСТЭМА

//

void sys_init(AirportSystem

*sys)

// Ініцыялізацыя пустой

сістэмы.

```
{
    sys->count      = 0;
    sys->searchTree = NULL;
    stack_init(&sys->history);
    queue_init(&sys->pendingQueue);
}
```

void sys_free(AirportSystem

*sys)

// Вызваленне ўсёй

памяці сістэмы.

```
{
    stack_free(&sys->history);
    queue_free(&sys->pendingQueue);
    bst_free(sys->searchTree);
    sys->searchTree = NULL;
    sys->count      = 0;
}
```

void sys_add_flight(AirportSystem *sys, Flight

f)

// Даданне рэйса ў сістэму.

```
{
```



```

    if (sys->count >= MAX_FLIGHTS)
    {
        printf("  Error: flight database is full (%d max).\n",
MAX_FLIGHTS);
        return;
    }
    sys->flights[sys->count++] = f;
    sys->searchTree = bst_insert(sys->searchTree,
f); // Адначасова ўстаўляем у BST.
}

Boolean sys_find_by_number(AirportSystem *sys, const char *num, int
*outIdx) // Пошук індэксу рэйса ў масіве.
{
    for (int i = 0; i < sys->count; i++)
        if (strcmp(sys->flights[i].flightNumber, num) == 0)
        {
            if (outIdx) *outIdx = i;
            return TRUE;
        }
    return FALSE;
}

void sys_delete_flight(AirportSystem *sys, int
idx) // Выдаленне рэйса і захаванне ў гісторыі.
{
    if (idx < 0 || idx >= sys->count) return;
    stack_push(&sys->history, sys-
>flights[idx]); // Захоўваем у стэк для
магчымага адмены.
    sys->searchTree = bst_delete(sys->searchTree,
                                sys->flights[idx].flightNumber);
    for (int i = idx; i < sys->count - 1; i++)
        sys->flights[i] = sys->flights[i + 1];
    sys->count--;
}

void sys_update_status(AirportSystem *sys, int idx, FlightStatus
newStatus) // Абнаўленне статусу рэйса.
{
    if (idx < 0 || idx >= sys->count) return;
    stack_push(&sys->history, sys-
>flights[idx]); // Захоўваем стан перад
змяненнем.
    sys->flights[idx].status = newStatus;
    sys->searchTree = bst_insert(sys->searchTree, sys-
>flights[idx]); // Абнаўляем запіс у BST.
}

```

```

void sys_sort_by_schedule(AirportSystem
*sys) // Сартаванне ўстаўкай па часе
раскладу.
{
    for (int i = 1; i < sys->count; i++)
    {
        Flight key = sys->flights[i];
        int j = i - 1;
        while (j >= 0 && datetime_compare(sys-
>flights[j].scheduleTime,
                                key.scheduleTime) > 0)
        {
            sys->flights[j + 1] = sys->flights[j];
            j--;
        }
        sys->flights[j + 1] = key;
    }
}

void sys_print_all(AirportSystem
*sys) // Вывад усіх рэйсаў у
таблічным выглядзе.
{
    if (sys->count == 0) { printf("\n No flights in the system.\n");
return; }
    flight_print_header();
    for (int i = 0; i < sys->count; i++)
        flight_print_row(i, &sys->flights[i]);
    printf("\n Total: %d flight(s).\n", sys->count);
}

void sys_search(AirportSystem *sys, const SearchFilter
*filter) // Пошук і вывод рэйсаў па фільтры.
{
    int found = 0;
    flight_print_header();
    for (int i = 0; i < sys->count; i++)
        if (flight_matches_filter(&sys->flights[i], filter))
        {
            flight_print_row(i, &sys->flights[i]);
            found++;
        }
    printf("\n Found: %d flight(s).\n", found);
}

void sys_undo(AirportSystem
*sys) // Адмяненне апошняга

```

дзеяння праз стэк.

```
{
    Flight last;
    if (!stack_pop(&sys->history, &last))
    {
        printf("  Nothing to undo.\n");
        return;
    }
    int
idx;
//
Перазапіс запісу калі існуе, іначай дадаем зноў.
    if (sys_find_by_number(sys, last.flightNumber, &idx))
        sys->flights[idx] = last;
    else
        sys_add_flight(sys, last);

    sys->searchTree = bst_insert(sys->searchTree, last);
    printf("  Undo successful. Restored flight %s.\n",
last.flightNumber);
}

//
=====
//  ВВОД
//
=====

void input_keyboard(AirportSystem
*sys) // Увод рэйсаў з клавiятуры.
{
    int n;
    printf("How many flights to enter: ");
    GetInt(&n);
    for (int i = 0; i < n; i++)
    {
        printf("\n--- Flight %d ---\n", i + 1);
        Flight f = {0};
        flight_input_keyboard(&f);
        sys_add_flight(sys, f);
    }
    printf("  %d flight(s) added.\n", n);
}

void input_txt_file(AirportSystem
*sys) // Загрузка рэйсаў з txt
файла.
{
    char filename[MAX_FILENAME];
```

```

printf("Enter filename (.txt): ");
read_line(filename, MAX_FILENAME, stdin);

FILE *f = fopen(filename, "r");
if (!f)
{
    printf("  Error: cannot open file '%s'.\n", filename);
    return;
}

char line[512];
int loaded = 0;
while (fgets(line, sizeof(line),
f)) // Пропускаем заголовки (рядки з
'#' або пустя).
{
    line[strcspn(line, "\n")] = '\0';
    if (line[0] == '#' || line[0] == '\0') continue;
    Flight fl = {0};
    if (flight_parse_line(line, &fl))
    {
        sys_add_flight(sys, fl);
        loaded++;
    }
    else
    {
        printf("  Warning: skipped unparseable line: %s\n", line);
    }
}
fclose(f);
printf("  Loaded %d flight(s) from '%s'.\n", loaded, filename);
}

void input_binary_file(AirportSystem
*sys) // Загрузка рейсаў з бінарнага
файла.
{
    char filename[MAX_FILENAME];
    printf("Enter filename (.bin): ");
    read_line(filename, MAX_FILENAME, stdin);

    FILE *f = fopen(filename, "rb");
    if (!f)
    {
        printf("  Error: cannot open binary file '%s'.\n", filename);
        return;
    }
}

```

```

    char magic[5] =
{0}; // Правляем
магічнае слова файла.
    if (fread(magic, 1, 4, f) != 4 || strcmp(magic, "APT1") != 0)
    {
        printf("  Error: invalid binary file format (bad magic).
Please choose another input option.\n");
        fclose(f);
        return;
    }

    int count = 0;
    if (fread(&count, sizeof(int), 1, f) != 1 || count <= 0 || count >
MAX_FLIGHTS)
    {
        printf("  Error: corrupted flight count in binary file. Please
choose another input option.\n");
        fclose(f);
        return;
    }

    int loaded = 0;
    for (int i = 0; i < count; i++)
    {
        Flight fl = {0};
        size_t r = fread(&fl, sizeof(Flight), 1, f);
        if (r != 1)
        {
            printf("  Warning: file ended prematurely after %d
flight(s).\n", loaded);
            break;
        }
        sys_add_flight(sys, fl);
        loaded++;
    }
    fclose(f);
    printf("  Loaded %d flight(s) from binary file '%s'.\n", loaded,
filename);
}

void input_choose(AirportSystem
*sys) // Меню выбару крыніцы
ўводу.
{
    while (1)
    {
        printf("\n=== Choose Input Method ===\n");
        printf("  1. Keyboard\n");

```

```

printf(" 2. Text file (.txt)\n");
printf(" 3. Binary file (.bin)\n");
printf(" 0. Cancel\n");
printf("Choice: ");
int ch; GetInt(&ch);
switch (ch)
{
    case 1: input_keyboard(sys);    return;
    case 2: input_txt_file(sys);    return;
    case 3: input_binary_file(sys); return;
    case 0:                                return;
    default: printf(" Invalid choice.\n");
}
}

//
=====
//  Вывод у файлы
//
=====

void output_txt_file(AirportSystem *sys, const char
*filename)          // Захаванне ўсіх рэйсаў у txt файл.
{
    FILE *f = fopen(filename, "w");
    if (!f) { printf(" Error: cannot open '%s' for writing.\n",
filename); return; }

    fprintf(f, "# Airport Flight Database -- %d flight(s)\n", sys-
>count);
    fprintf(f, "# Format: FlightNum Type Airline City Aircraft "
        "Sched(DD.MM.YYYY/HH:MM) Actual(DD.MM.YYYY/HH:MM)
Status Terminal Gate\n");

    for (int i = 0; i < sys->count; i++)
    {
        const Flight *fl = &sys->flights[i];
        char statusStr[15];
        strcpy(statusStr, flight_status_str(fl->status));
        for (int j = 0; statusStr[j]; j+
+)                                // Замяняем пробелы на
падкарэсліванні для аднаслоўнага поля.
            if (statusStr[j] == ' ') statusStr[j] = '_';

        fprintf(f, "%-9s %-3s %-20s %-22s %-20s %02d.%02d.%04d/%02d:
%02d %02d.%02d.%04d/%02d:%02d %-10s %c %d\n",
            fl->flightNumber,

```

```

        flight_type_str(fl->type),
        fl->airline,
        fl->city,
        fl->airplaneModel,
        fl->scheduleTime.day, fl->scheduleTime.month, fl->
>scheduleTime.year,
        fl->scheduleTime.hour, fl->scheduleTime.minute,
        fl->actualTime.day, fl->actualTime.month, fl->
>actualTime.year,
        fl->actualTime.hour, fl->actualTime.minute,
        statusStr,
        fl->terminal,
        fl->gate);
    }
    fclose(f);
    printf(" Saved %d flight(s) to text file '%s'.\n", sys->count,
filename);
}

void output_binary_file(AirportSystem *sys, const char
*filename) // Захаванне ўсіх рэйсаў у бінарны файл.
{
    FILE *f = fopen(filename, "wb");
    if (!f) { printf(" Error: cannot open '%s' for writing.\n",
filename); return; }

    fwrite("APT1", 1, 4,
f); // Магічны загалоўак
файла.
    fwrite(&sys->count, sizeof(int), 1,
f); // Колькасць рэйсаў.
    fwrite(sys->flights, sizeof(Flight), sys->count,
f); // Масіў рэйсаў.
    fclose(f);
    printf(" Saved %d flight(s) to binary file '%s'.\n", sys->count,
filename);
}

void output_choose(AirportSystem
*sys) // Меню выбару фармату
вываду.
{
    printf("\n=== Current Flight Board
===\n"); // Заўсёды паказваем на экран.
    sys_print_all(sys);

    printf("\n=== Save to File ===\n");
    printf(" 1. Save as text file (.txt)\n");
}

```

```

printf(" 2. Save as binary file (.bin)\n");
printf(" 0. Don't save\n");
printf("Choice: ");
int ch; GetInt(&ch);
if (ch == 0) return;

char filename[MAX_FILENAME];
printf("Enter filename: ");
read_line(filename, MAX_FILENAME, stdin);

if (ch == 1) output_txt_file(sys, filename);
else if (ch == 2) output_binary_file(sys, filename);
else printf(" Invalid choice, not saving.\n");
}

//
=====
// АЎТЭНТЫФІКАЦЫЯ
//
=====

Boolean auth_login(User
*out) // Простая
аўтэнтыфікацыя ва ўбудаваную базу.
{
    static const User DB[] =
    { // Убудаваныя ўліковыя
        запісы для дэманстрацыі.
        {"admin", "admin123", ROLE_ADMIN},
        {"passenger", "pass", ROLE_PASSENGER}
    };
    static const int DB_SIZE = 2;

    char username[30], password[30];
    printf("Username: "); read_line(username, 30, stdin);
    printf("Password: "); read_line(password, 30, stdin);

    for (int i = 0; i < DB_SIZE; i++)
        if (strcmp(DB[i].username, username) == 0 &&
            strcmp(DB[i].password, password) == 0)
        {
            *out = DB[i];
            return TRUE;
        }
    return FALSE;
}

//

```



```

=====
//  МЕНЮ — ПАДМЕНЮ ПОШУКУ
//
=====

void menu_search(AirportSystem
*sys)                                     // Інтерактыўная пабудова
фільтра і пошук.
{
    SearchFilter filter;
    memset(&filter, 0, sizeof(filter));

    printf("\n=== Search Flights ===\n");

    printf("Filter by type?      (1=yes, 0=no): ");
    int b; GetInt(&b);
    if (b) {
        filter.useTypeFilter = TRUE;
        printf("  0=Arrival, 1=Departure: ");
        GetInt(&b);
        filter.targetType = (b == 1) ? DEPARTURE : ARRIVAL;
    }

    printf("Filter by city?      (1=yes, 0=no): ");
    GetInt(&b);
    if (b) {
        filter.useCityFilter = TRUE;
        printf("  City name: ");
        read_line(filter.targetCity, 50, stdin);
    }

    printf("Filter by status?    (1=yes, 0=no): ");
    GetInt(&b);
    if (b) {
        filter.useStatusFilter = TRUE;
        printf("  0=On time  1=Delayed  2=Boarding  3=Departed\n4=Landed  5=Canceled\n");
        printf("  Status: "); GetInt(&b);
        if (b < 0 || b > 5) b = 0;
        filter.targetStatus = (FlightStatus)b;
    }

    printf("Filter by date/time (1=yes, 0=no): ");
    GetInt(&b);
    if (b) {
        filter.useTimeFilter = TRUE;
        printf("  Show flights from this time onward:\n");
        datetime_input(&filter.startTime);
    }
}

```

```

    }

    printf("\n--- Search Results ---\n");
    sys_search(sys, &filter);
}

//
=====
//  МЕНЮ — ЧАРГА РЭЙСАЎ НА ПАЦВЯРДЖЭННЕ
//
=====

void menu_pending_queue(AirportSystem
*sys)                                     // Кіраванне чаргой рэйсаў на
пацвярджэнне.
{
    while (1)
    {
        printf("\n=== Pending Queue (%d) ===\n", sys-
>pendingQueue.size);
        printf("  1. Add current flight to queue\n");
        printf("  2. Confirm (dequeue) next flight -> add to
board\n");
        printf("  3. View queue\n");
        printf("  0. Back\n");
        printf("Choice: ");
        int ch; GetInt(&ch);

        if (ch == 0) return;

        if (ch == 1)
        {
            Flight f = {0};
            printf("\n--- Enter flight for queue ---\n");
            flight_input_keyboard(&f);
            queue_enqueue(&sys->pendingQueue, f);
            printf("  Flight %s added to pending queue.\n",
f.flightNumber);
        }
        else if (ch == 2)
        {
            Flight f;
            if (queue_dequeue(&sys->pendingQueue, &f))
            {
                sys_add_flight(sys, f);
                printf("  Flight %s confirmed and added to the board.
\n", f.flightNumber);
            }
        }
    }
}

```

```

        else printf(" Queue is empty.\n");
    }
    else if (ch == 3)
    {
        queue_print(&sys->pendingQueue);
    }
}

//
=====
//  МЕНЮ — BST ДРЭВА
//
=====

void menu_bst(AirportSystem
*sys)                                     // Показ BST у парадку
ўзрастання нумара.
{
    printf("\n=== Flights sorted by number (BST in-order) ===\n");
    if (sys->searchTree == NULL) { printf(" (tree is empty)\n");
return; }
    flight_print_header();
    bst_inorder_print(sys->searchTree);

    printf("\n--- BST Quick Search ---\n");
    printf("Enter flight number to search (or '.' to skip): ");
    char num[10];
    read_line(num, 10, stdin);
    if (strcmp(num, ".") != 0)
    {
        TreeNode *found = bst_search(sys->searchTree, num);
        if (found)
        {
            printf(" Found:\n");
            flight_print_header();
            flight_print_row(0, &found->data);
        }
        else printf(" Flight '%s' not found in BST.\n", num);
    }
}

//
=====
//  МЕНЮ — ДАДАЦЬ РЭЙС (АДМІН)
//
=====

```

```

void menu_add(AirportSystem
*sys)                                     // Дадаць адзін рэйс
непасрэдна.
{
    Flight f = {0};
    printf("\n=== Add New Flight ===\n");
    flight_input_keyboard(&f);
    sys_add_flight(sys, f);
    printf("  Flight %s added successfully.\n", f.flightNumber);
}

//
=====
//  МЕНЮ — РЭДАГАВАЦЬ РЭЙС (АДМІН)
//
=====

void menu_edit(AirportSystem
*sys)                                     // Рэдагаванне існуючага
рэйса.
{
    if (sys->count == 0) { printf("  No flights to edit.\n");
return; }
    sys_print_all(sys);

    printf("Enter flight number to edit: ");
    char num[10];
    read_line(num, 10, stdin);

    int idx;
    if (!sys_find_by_number(sys, num, &idx))
    {
        printf("  Flight '%s' not found.\n", num);
        return;
    }

    printf("  What to edit?\n");
    printf("  1. Status only\n");
    printf("  2. Full re-entry\n");
    printf("  0. Cancel\n");
    printf("Choice: ");
    int ch; GetInt(&ch);

    if (ch == 1)
    {
        printf("  0=On time  1=Delayed  2=Boarding  3=Departed
4=Landed  5=Canceled\n");
        printf("  New status: ");
    }
}

```

```

        int st; GetInt(&st);
        if (st < 0 || st > 5) { printf(" Invalid status.\n");
return; }
        sys_update_status(sys, idx, (FlightStatus)st);
        printf(" Status updated.\n");
    }
    else if (ch == 2)
    {
        stack_push(&sys->history, sys-
>flights[idx]); // Захоўваем бягучы стан для undo.
        sys->searchTree = bst_delete(sys->searchTree,
                                     sys->flights[idx].flightNumber);
        flight_input_keyboard(&sys->flights[idx]);
        sys->searchTree = bst_insert(sys->searchTree, sys-
>flights[idx]);
        printf(" Flight updated.\n");
    }
}

//
=====
//  МЕНЮ — ВЫДАЛИЦЬ РЭЙС (АДМІН)
//
=====

void menu_delete(AirportSystem
*sys) // Выдаленне рэйса са
стэкам гісторыі.
{
    if (sys->count == 0) { printf(" No flights to delete.\n");
return; }
    sys_print_all(sys);

    printf("Enter flight number to delete: ");
    char num[10];
    read_line(num, 10, stdin);

    int idx;
    if (!sys_find_by_number(sys, num, &idx))
    {
        printf(" Flight '%s' not found.\n", num);
        return;
    }
    sys_delete_flight(sys, idx);
    printf(" Flight '%s' deleted (can be undone).\n", num);
}

//

```

```

=====
//  ГАЛОЎНАЕ МЕНЮ
//
=====

void menu_main(AirportSystem *sys, User
*user)                                // Галоўнае меню праграмы.
{
    while (1)
    {
        printf("\n===== \n");
        printf("      AIRPORT INFORMATION SYSTEM      \n");
        printf("===== \n");
        printf("  User: %s  [%s]\n",
            user->username,
            user->role == ROLE_ADMIN ? "ADMIN" : "PASSENGER");
        printf("  Flights on board: %d | Pending: %d | History
stack: %d\n\n",
            sys->count, sys->pendingQueue.size, sys->history.size);

        printf("  1. View all flights\n");
        printf("  2. Search / filter flights\n");
        printf("  3. Sort by schedule\n");
        printf("  4. BST index (search by number)\n");

        if (user->role == ROLE_ADMIN)
        {
            printf("    == Admin ==\n");
            printf("  5. Add flight\n");
            printf("  6. Edit flight\n");
            printf("  7. Delete flight\n");
            printf("  8. Undo last change\n");
            printf("  9. Pending queue management\n");
            printf(" 10. Load more data (input)\n");
        }

        printf("    == File ==\n");
        printf(" 11. Save / export\n");
        printf("   0. Exit\n");
        printf("\nChoice: ");
        int ch; GetInt(&ch);

        switch (ch)
        {
            case 0:
                printf("Goodbye!\n");
                return;

```

```

case 1:
    sys_print_all(sys);
    press_enter();
    break;

case 2:
    menu_search(sys);
    press_enter();
    break;

case 3:
    sys_sort_by_schedule(sys);
    printf("  Flights sorted by scheduled time.\n");
    sys_print_all(sys);
    press_enter();
    break;

case 4:
    menu_bst(sys);
    press_enter();
    break;

case 5:
    if (user->role != ROLE_ADMIN) goto no_access;
    menu_add(sys);
    press_enter();
    break;

case 6:
    if (user->role != ROLE_ADMIN) goto no_access;
    menu_edit(sys);
    press_enter();
    break;

case 7:
    if (user->role != ROLE_ADMIN) goto no_access;
    menu_delete(sys);
    press_enter();
    break;

case 8:
    if (user->role != ROLE_ADMIN) goto no_access;
    sys_undo(sys);
    press_enter();
    break;

case 9:
    if (user->role != ROLE_ADMIN) goto no_access;

```

```

        menu_pending_queue(sys);
        break;

    case 10:
        if (user->role != ROLE_ADMIN) goto no_access;
        input_choose(sys);
        press_enter();
        break;

    case 11:
        output_choose(sys);
        press_enter();
        break;

    default:
        printf("    Unknown option.\n");
        break;

    no_access:
        printf("    Access denied. Admin only.\n");
        press_enter();
        break;
}
}
}

```

Листинг 2 — Исходный код файла реализации func.c

ПРИЛОЖЕНИЕ Д

(обязательное)

Код файла main.c

```
// Праграма "Аэрапорт": сістэма кіравання рэйсамі.
// Структуры дадзеных: стэк (гісторыя змяненняў/undo),
//                                     чарга (рэйсы на пацвярджэнне),
//                                     бінарнае дрэва пошуку (хуткі пошук па нумары).
// Увод: клавіятура / txt-файл / бінарны файл.
// Вывад: экран + txt-файл або бінарны файл.

#include "header.h"

int main(void)
{
    AirportSystem sys;

    sys_init(&sys); // Ініцыялізацыя пустой сістэмы.

    printf("=====\n");
    printf("      AIRPORT INFORMATION SYSTEM v1.0      \n");
    printf("=====\\n\\n");

    User
    currentUser; // Аўтэнтыфікацыя карыстальніка.
    int attempts = 0;
    printf("=== Login ===\n");
    printf(" (admin/admin123 or passenger/pass)\n\\n");
    while (!auth_login(&currentUser))
    {
        attempts++;
        if (attempts >= 3)
        {
            printf(" Too many failed attempts. Exiting.\n");
            sys_free(&sys);
            return 1;
        }
        printf(" Invalid credentials. Try again (%d/3).\n",
attempts);
    }
    printf("\\n Welcome, %s!\n", currentUser.username);

    printf("\\n=== Initial Data Load
===\n"); // Першапачатковая загрузка
дадзеных.
    input_choose(&sys);
```

```

    menu_main(&sys,
&currentUser); // Галоўнае меню
праграмы.

    if (sys.count >
0) // Прапанова
захаваць перад выхадам.
    {
        printf("\nSave data before exit? (1=yes, 0=no): ");
        int save; GetInt(&save);
        if (save) output_choose(&sys);
    }

sys_free(&sys); //
Вызваляем усю памяць перад выхадам.
    return 0;
}

```

Листинг 3 — Исходный код главного файла программы main.c